Governing Human–AI Collaborative Project Workflows in Multi-Platform Digital Delivery: A Design Science Study of Auditable Handoffs and Idempotent Release Operations

HuaiCheng Sheng[1,*]

[1]Agentic Work Lab, Hangzhou Chengxin Holding Group Co., Ltd., Hangzhou, Zhejiang, China

[*]Corresponding author. E-mail: `ZackSheng@gmail.com` | ORCID: `0009-0003-2900-6377` | GitHub: `https://github.com/whitecarp`

ABSTRACT

Context: When large language model (LLM) agents move from conversation to execution, project governance must control actions on external platforms that cannot be undone. An uncertain submission that is retried can be published twice, and descriptive metrics are easily reported as evidence of AI-driven gains.

Objective: Following Design Science Research (DSR) and stage-gate project control theory, we design and evaluate a human-in-the-loop (HITL) governance framework for multi-agent, multi-platform digital delivery, implemented in two open-source artifacts, *Agentic Work Lab* and *PublishOps CrossCast*.

Method: The framework has four layers: (1) a deterministic owner, stop-flag and due-time gate (`planRun`); (2) binding of each release command to the SHA-256 digest of the reviewed plan, with SHA-256 media checks; (3) a receipt state machine that records an `UNKNOWN` state before the single final submission and never re-submits automatically; and (4) independent read-back with escalation to a person. We evaluate it in a nine-day field deployment (26 October–3 November 2026) in one creator–engineering laboratory, using a 69-record operational journal, a scheduled heartbeat trace, a five-platform release of one film, 2,737 offline test functions, 30 SHA-256-recorded video versions (2.21 GB) and a 55-record channel snapshot.

Results: The journal records one platform write and no replayed upload. The heartbeat started 24.992 s late and ran 31.175 s without task errors. When Douyin's read-back timed out, the receipt stayed `UNKNOWN`, nothing was re-submitted, and the owner confirmed the work by phone. In one instrumented Instagram run, 380.551 s of the 412.217 s publish command (92.32%) was the wait after the single Share, spanning transfer, processing and confirmation.

Conclusions: Here, agentic delivery was governed by deterministic gates, content-bound approvals and at-most-once submission with human reconciliation rather than by prompt instructions. The evidence is single-case and descriptive; code, data and a script that recomputes every reported figure are public.

Keywords: Project Management Governance; Human–AI Collaboration; Multi-Agent Workflows; Design Science Research; Idempotent Release Engineering; Stage-Gate Control; PublishOps

Research Highlights

- A four-layer stage-gate design governs human–AI delivery to five platforms
- Receipts are saved as UNKNOWN before the one final submit; no auto re-submit
- A 69-record field journal shows one platform write and no replayed upload
- In one Instagram run, 92.32% of the publish command was the wait after Share
- Descriptive channel data are kept apart from causal claims about AI output

1. Introduction

The rapid maturation of Large Language Models (LLMs) and tool-using agents is changing how projects in software engineering and digital operations are run (Holzmann et al., 2022; Müller et al., 2024; Wang et al., 2024). Unlike automation scripts that execute fixed subroutines within narrow integration boundaries, agentic workflows deploy heterogeneous foundation models—reasoning-oriented coding agents, interface construction agents and scheduled monitoring agents—to collaborate on multi-stage deliverables (Wu et al., 2024; Hong et al., 2024; Qian et al., 2024). In digital media production, release engineering and outreach operations, such multi-agent arrangements promise less manual coordination and shorter turnaround (Raisch & Krakowski, 2021; Berente et al., 2021).

Deploying autonomous or semi-autonomous agents into production, however, exposes governance problems that classical project management methods (PMI, 2021; Kerzner, 2022) and conversational multi-agent frameworks were not designed to address. Four failure modes can arise when LLM agents act on shared repositories and external platforms:

1. **Multi-Agent Handoff Drift and Role Ambiguity:** When several specialized models (here, GPT/Codex for scripts, code-driven video and backend verification; Claude for interface construction; and DeepSeek for scheduled monitoring) work asynchronously on shared project state, context boundaries can degrade. Without ownership locks and explicit acceptance gates, an agent woken by a schedule may repeat a task another instance has claimed, overwrite approved revisions or act on a stale checklist (Whyte et al., 2016; Qian et al., 2024).
2. **Non-Idempotent External Side Effects under Platform Uncertainty:** Release operations across Bilibili, YouTube, X, Douyin and Instagram mix official APIs with stateful browser sessions. Network interruptions, transcoding queues, verification challenges or read-back timeouts can leave a submission in an indeterminate (`UNKNOWN`) state (Schneider, 1990; Kleppmann, 2017). Unlike payment and cloud APIs, these web submission flows offer no idempotency key that would let the platform discard a duplicate (Helland, 2012), so a retry loop that reads a missing confirmation as failure may publish twice, disturb playlist order or trigger rate limits.
3. **Opaque Quality Degradation in Generative Deliverables:** One-shot generative pipelines can introduce localized defects—single-frame flashes before transitions, audio–video drift, clipped audio or mobile overlay cropping—that pass superficial completion checks. Without editable intermediate representations and signal-level checks, repairing one defect means regenerating the whole deliverable and discarding validated segments.
4. **Epistemic Confounding in Project Performance Evaluation:** AI adoption is often reported with claims of labour savings or audience growth that the underlying data cannot support (Raisch & Krakowski, 2021). Comparing historical human-led output with a newly launched AI-assisted series confounds exposure age (years of cumulative traffic versus days), topic and platform recommendation dynamics, and counting generated drafts as delivered work hides the human review and rework that acceptance required.

To address these problems, this study adopts Design Science Research (DSR) (Hevner et al., 2004; Peffers et al., 2007; Gregor & Hevner, 2013; Venable et al., 2016) to build and evaluate a human-in-the-loop (HITL) governance framework and release operations system: **Agentic Work Lab** and its standalone multi-platform release engine **PublishOps CrossCast** (`vpub`). Rather than treating AI governance as prompt engineering, we formulate multi-agent delivery as a verifiable stage-gate control system (Cooper, 1990; Conforto & Amaral, 2016; Bianchi et al., 2022) backed by content digests, frame-level quality checks and fault-tolerant state transitions.

Specifically, this paper investigates three research questions (RQs):

- **RQ1 (Governance Architecture & Handoff Control):** How can stage-gate project management principles be formalized into deterministic software invariants to govern multi-model handoffs (GPT/Codex, Claude and DeepSeek) and prevent unauthorized or duplicate autonomous actions?
- **RQ2 (Idempotent Release Engineering & Fault Recovery):** How can a persistent receipt state machine coordinate multi-platform releases, in-place revisions and authorized deletions across API and browser-automated channels when post-submission verification meets timeouts or human authentication barriers?
- **RQ3 (Empirical Latency, Quality & Epistemic Validation):** What operational characteristics, latency bottlenecks, defect-repair patterns and methodological boundaries are observed when this architecture is used in a nine-day field deployment across five platforms?

The contributions are fivefold:

1. **A Four-Layer Agentic Project Governance Framework:** We synthesize stage-gate control (Cooper, 1990), configuration management (Whyte et al., 2016) and human-centered automation (Parasuraman et al., 2000; Shneiderman, 2020) with commit-protocol reasoning from distributed systems (Section 2.4) into four enforceable layers: (i) a sole-operator and temporal scheduling gate, (ii) plan-hash and media-integrity binding, (iii) an idempotent receipt state machine with `UNKNOWN`-state suppression, and (iv) independent read-back, in-place revision and HITL handoff.
2. **Algorithms with Stated Assumptions:** Three algorithms govern job planning (Algorithm 1), publication with a receipt persisted before the final submission (Algorithm 2), and identity-bound revision and consent-bound deletion (Algorithm 3); Proposition 1 states the conditions under which automated re-submission cannot occur.
3. **Open-Source Reference Implementation:** We describe the architecture and evolution of *PublishOps CrossCast* from `v0.2.2` to commit `75b7abc` (PR #30), whose test suite grew from 2,009 to 2,737 test functions between the `v0.16.0` release and that commit, together with dependency-free Node.js verification scripts in *Agentic Work Lab*.

4. **Field Evidence and a Latency Profile:** We report the system's behaviour in a 69-record operational journal, 30 SHA-256-recorded video versions (2.21 GB), a 13,034-frame defect scan, a crash-recovery check on a private upload, and a five-platform release in which one in-doubt submission was held without re-submission. In one Instagram publication, the wait after Share accounted for 92.32% of the publish command, which motivates persisting submission state before that wait.
5. **A Protocol for Auditable AI Project Reporting:** We demonstrate reporting rules that keep observational receipts apart from causal inference, guarding against exposure-age confounding when human-led (n = 45) and AI-assisted (n = 10) cohorts are compared.

The study is a single-case design science evaluation: its claims concern how the mechanisms behaved and why, not how often such failures occur across projects (Section 7.4).

2. Related Work and Theoretical Background

2.1 Project Management Governance and Stage-Gate Control in the AI Era

Classical project management emphasizes structured control mechanisms—Work Breakdown Structures, configuration management and Stage-Gate reviews—to manage technical uncertainty and enforce accountability across project phases (Cooper, 1990; Whyte et al., 2016; PMI, 2021; Kerzner, 2022). In Cooper's Stage-Gate model and in hybrid agile–stage-gate frameworks (Conforto & Amaral, 2016; Bianchi et al., 2022), iterative work within a stage is separated from downstream commitment by decision gates, each with predefined deliverables, explicit criteria and a gatekeeper who decides whether to proceed, recycle, hold or stop. Project governance research separates the framework that assigns decision rights and accountability from the management of execution itself (Too & Weaver, 2014); in delivery terms, governance decides who may authorize an irreversible step, and on what evidence.

Recent scholarship examines how artificial intelligence changes project governance, decision authority and human–machine agency (Holzmann et al., 2022; Müller et al., 2024). Raisch & Krakowski (2021) describe the tension between automation and augmentation in management: removing human judgment from critical loops invites drift and brittle failure under novel cases, whereas isolated augmentation fails to scale. Work on human–automation interaction argues that reliable socio-technical systems combine high automation with high human control through transparent audit trails, explicit pre-authorization and graceful exception handoffs (Parasuraman et al., 2000; Amershi et al., 2019; Shneiderman, 2020). Much of the project management literature on AI, however, relies on conceptual typologies or expert Delphi surveys (Holzmann et al., 2022); translating stage-gate governance into software-enforced invariants for live, multi-model agents remains largely unaddressed.

2.2 Multi-Agent LLM Workflows and Coordination Vulnerabilities

Reasoning-and-acting paradigms such as ReAct (Yao et al., 2023) and Reflexion (Shinn et al., 2023) have led to multi-agent frameworks including AutoGen (Wu et al., 2024), MetaGPT (Hong et al., 2024) and ChatDev (Qian et al., 2024). MetaGPT encodes Standardized Operating Procedures into role-specific prompts (product manager, architect, engineer, QA), while ChatDev structures software development as a sequential chat chain. These frameworks perform well on self-contained coding benchmarks (Wang et al., 2024), but continuous multi-platform operations expose three gaps:

- **No Cross-Process Execution Locks:** Prompt-based procedures do not prevent two agent sessions (for example, an interactive coding session and a scheduled heartbeat) from mutating the same browser profile or checklist at once.
- **No Binding of Approval to Artifact Content:** In conversational handoffs, an agent may approve a text summary while the binary artifact on disk is an older, unpatched version or has been misread.
- **Unbounded Retries on External Timeouts:** Tool-calling loops commonly treat timeouts or selector failures as transient errors and invoke the tool again, which duplicates a publication whenever the platform had in fact accepted the first submission.

2.3 DevOps, Release Engineering, and Idempotent State Machines

In software engineering, Continuous Integration and Continuous Delivery (CI/CD) pipelines reduce deployment risk through automated build verification, immutable artifact digests and staged rollouts (Humble & Farley, 2010; Bass et al., 2015; Adams & McIntosh, 2016; Shahin et al., 2017; Leite et al., 2019). A cornerstone of fault-tolerant distributed systems and transaction processing is *idempotency*: repeating an operation, or recovering from an interrupted request, must not produce duplicate side effects (Schneider, 1990; Gray & Reuter, 1992; Kleppmann, 2017). Payment and cloud APIs commonly accept a client-chosen idempotency key so that the server can discard duplicates (Helland, 2012); browser-driven submission to consumer platforms offers no such key,

so the client alone must decide whether a second attempt is safe. Table 1 contrasts multi-agent frameworks, CI/CD pipelines and generic upload scripts with the proposed *Agentic Work Lab* and *PublishOps CrossCast* architecture across eight governance dimensions.

Table 1: Comparative analysis of multi-agent frameworks, release pipelines and the proposed governance architecture across eight governance dimensions. Entries describe built-in mechanisms as documented; "No" means not provided as a built-in mechanism, not that it cannot be built on top. Upload helpers are characterized generically.

Framework / System	Multi-Model Handoffs	Temporal & Owner Gate	Crypto Plan & SHA-256 Lock	Hybrid API + Pipe Browser	`UNKNOWN` Latch (Zero Retry)	Identity-Bound Edit/Delete	Epistemic & Non-Causal Audit
AutoGen (Wu et al., 2024)	Conversational chat loops	No native lock	No	External tools only	No (standard retry)	No	No
MetaGPT (Hong et al., 2024)	Prompt-level SOP roles	Sequential memory	No	No	No	No	No
ChatDev (Qian et al., 2024)	Dual-agent chat chain	Phase-level only	Git commit only	No	No	No	No
Standard CI/CD (Humble & Farley, 2010)	Static pipeline stages	Runner queue locks	Artifact SHA-256	REST APIs only	API-dependent	Rollback scripts	Build logs only
Open-Source Upload Helpers (generic)	Single-script execution	Cron timer only	No plan-hash lock	Browser form fill	No (risks double post)	No read-back verification	No
Agentic Work Lab & CrossCast (Ours)	**File-mediated state handoffs (GPT/Claude/DS)**	**Deterministic `planRun` + OS lock**	**`--confirm` Plan Hash + Stream SHA-256**	**Browser sessions + `bili-cli`; API routes in a separate add-on**	**Receipt saved as `UNKNOWN` before the one final press**	**`--verify-only` + Absence Read-Back**	**Enforced machine-readable limits**

2.4 Synthesis: Reading Stage-Gate Control as a Commit Protocol

We read the two literatures together through one observation: a stage gate and a distributed commit protocol solve the same problem, namely how to make an irreversible step happen at most once, on the right object and under the right authority, when parts of the system may fail or disagree. The last row of Table 1 is the resulting design; the correspondence below states it as five design propositions that Section 4 implements and Section 6 examines.

- **Gatekeeping as admission control.** A stage gate admits work only when its owner, state and criteria allow (Cooper, 1990; Too & Weaver, 2014). In software, the same decision becomes a pure function of the operating context that returns *skip*, *no work due* or a plan, never an action (Layer 1).
- **Configuration baselines as content addresses.** Configuration management fixes a reviewed baseline so that what is delivered equals what was approved (Whyte et al., 2016). Hashing the reviewed plan and the media, and requiring that digest to run the command, turns approval into a binding on content rather than on a description of it (Layer 2).
- **The go decision as an at-most-once commit.** Where the receiving platform cannot discard duplicates, exactly-once submission is out of the client's reach; the client can only choose between at-least-once (retry) and at-most-once (never retry) behaviour (Helland, 2012; Kleppmann, 2017). Recording the intent to submit durably before acting, as a write-ahead log does (Gray & Reuter, 1992), selects at-most-once and makes every uncertain outcome visible (Layer 3).
- **Post-gate review as an end-to-end check.** Only the endpoint can confirm that an effect occurred (Saltzer et al., 1984); a local log shows only that a command was sent. Read-back from the platform is therefore the evidence that closes a gate (Layer 4).
- **Escalation as resolution of in-doubt transactions.** In two-phase commit, a participant that loses contact after voting is *in doubt* and cannot decide alone (Gray & Reuter, 1992). An `UNKNOWN` receipt is the same state, and resolving it is a human decision—which also leaves people with exactly the residual cases automation cannot settle (Bainbridge, 1983). Compensation by deleting and reposting, the usual saga remedy (Garcia-Molina & Salem, 1987), is unsafe here because a public post carries replies, an identifier and reach that a repost cannot restore.

3. Research Methodology

3.1 Design Science Research (DSR) Framework

We follow the six-step Design Science Research Methodology of Peffers et al. (2007) and the evaluation guidance of Hevner et al. (2004), Gregor & Hevner (2013) and Venable et al. (2016):

1. **Problem Identification and Motivation:** Uncontrolled multi-agent handoffs, duplicate external writes during uncertain platform states, localized defects in generative media, and confounded efficiency claims in AI-assisted delivery (Section 1).
2. **Definition of Solution Objectives:** A deterministic governance architecture that enforces sole-operator scheduling, binds approvals to artifact content, never re-submits automatically once a final submission may have occurred, supports targeted frame and audio repair, and keeps observational evidence apart from causal inference.
3. **Design and Development:** The four-layer governance model, its invariants and algorithms (Section 4), realized in two coupled open-source artifacts, *Agentic Work Lab* and *PublishOps CrossCast* (Section 5).
4. **Field Demonstration:** Use of the artifacts from 26 October to 3 November 2026 in one creator–engineering laboratory, spanning two episodic series (*GPT Duty Log* Ep 01–06 and *Whale Girl Shift Log* Ep 01–07), standalone experimental studies and the cross-platform short film *I Watched It For You (v3)*.
5. **Evaluation:** Following FEDS (Venable et al., 2016), we combine naturalistic evaluation (a 69-record operational journal, a scheduled heartbeat trace, a five-platform release and a 55-record channel snapshot) with artificial evaluation through offline regression tests (2,737 test functions in *CrossCast* at commit `75b7abc`, many of which simulate faults, and 10 workflow gate tests in *Agentic Work Lab*) (Section 6).
6. **Communication:** Publication of source code, SHA-256-recorded media deliverables (N = 30 versions, 2.21 GB), machine-readable JSON/CSV receipts and a script that recomputes every reported figure (`paper/reproduce_paper_tables.mjs`).

Researcher role. The author designed, operated and evaluated both artifacts. We therefore treat the study as researcher–practitioner intervention in the sense of action design research (Sein et al., 2011) and technical action research (Wieringa, 2014). Two choices limit the resulting bias: the evidence consists of machine-readable receipts written when each event happened rather than retrospective accounts, and every receipt, script and dataset is public so that each figure can be recomputed independently.

3.2 Epistemic Hygiene and Evidence Classification Protocol

A central methodological element of the study is an explicit epistemic hygiene protocol applied to all project documentation and data exports (Runeson & Höst, 2009). For every research topic and operational update, the laboratory records four elements: **(i) the original claim, (ii) visible primary evidence, (iii) an independent verification test, and (iv) applicable limits**. Specifically:

- **Separation of Observation and Causal Inference:** Platform observations (lifetime views, 24-hour completion rates, command durations) are stored with sampling timestamps and provenance notes. Causal inferences, such as asserting that an AI editing technique increased recommendations, are rejected unless backed by controlled, age- and topic-matched comparisons.
- **Granular State Differentiation:** Local preparation, dry-run rehearsal, sign-in preflight, submission, platform review and verified public visibility are distinct states. UI observations and API verifications occupy separate fields, and an unverified or timed-out read-back is recorded as `UNKNOWN` rather than coerced into success or failure.
- **Immutable Snapshot Preservation:** Repeated data collections are archived as separate observations (for example, the 28 October afternoon snapshot of 52 records is kept apart from the evening snapshot of 55 records and from the daily updates of 29 October–3 November) rather than overwritten.
- **Explicit Tagging of Unexecuted Plans:** Planned designs (such as a blinded identical-task comparison of models on merchant research) and unmeasured optimizations (such as the wall-clock gain of concurrent publication) stay labelled as unrun until live receipts exist.

3.3 Data Sources and Evidence Window

The evaluation draws on records with different sampling windows, which we keep separate: the operational journal (69 records; 46 timestamped between 26 October 01:27 and 27 October 02:45, UTC+8; exported on 28 October); the scheduled heartbeat trace (28 October); the channel snapshot (55 records sampled on 28 October at 21:04, UTC+8); the release record of *I Watched It For You (v3)*

(2–3 November); the media manifest (30 versions, GitHub assets checked on 3 November); and the *CrossCast* code at commit `75b7abc` (3 November). Both repositories have changed since; every figure refers to these files and commits.

4. The Four-Layer Human–AI Project Governance Architecture

To answer **RQ1** and **RQ2**, we formalize the project governance architecture as a four-layer hierarchical control system mediating between the human Project Manager, specialized LLM agents, local media/code artifacts, and external publishing platforms (Figure 1).

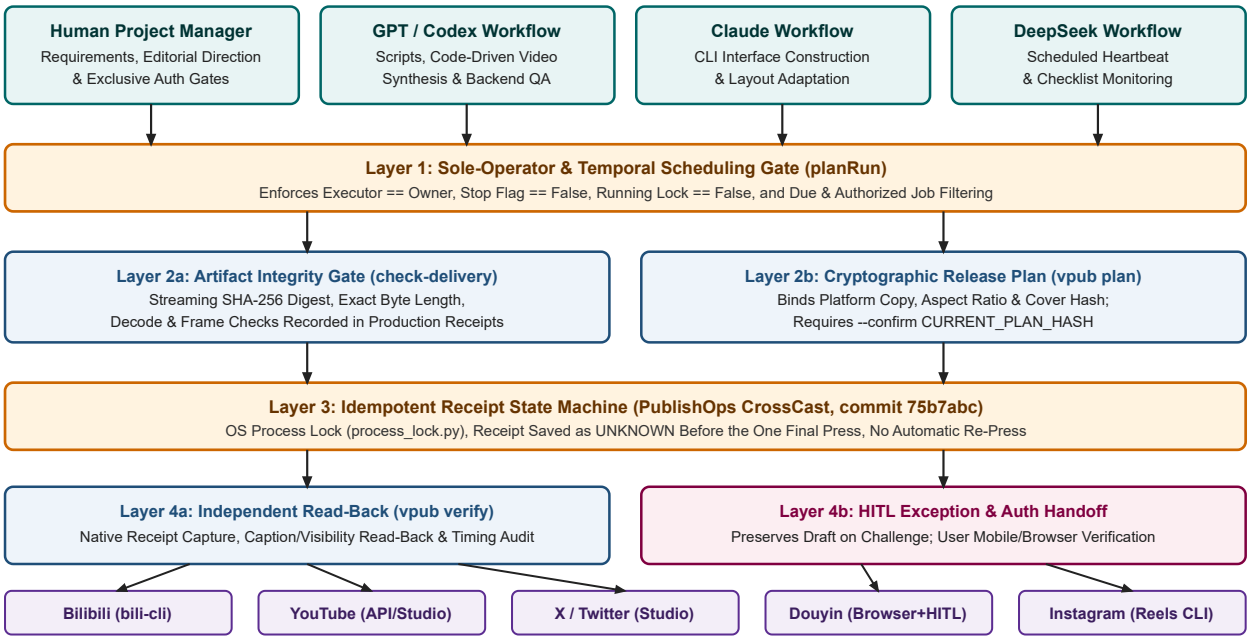


Figure 1. The four-layer human–AI collaborative project governance and multi-platform release architecture implemented in *Agentic Work Lab* and *PublishOps CrossCast*.

4.1 Multi-Agent Role Allocation and Handoff Boundaries

Within the laboratory environment, project execution is partitioned across four distinct actors: the Human Project Manager and three specialized AI agent workflows (GPT/Codex, Claude, and DeepSeek). Rather than allowing agents to communicate via unstructured conversational loops, all inter-agent handoffs are mediated through version-controlled state files (`operations-checklist.md`, `delivery-manifest.json`, and `production-update-*.json`). Table 2 defines the task scope, handoff artifacts, and operational constraints for each actor.

Table 2: Multi-agent role allocation, handoff artifacts, and governance boundaries across the project lifecycle.

Project Actor	Assigned Functional Scope	Observable Evidence & Handoff Artifacts	Enforced Governance & Safety Constraints
Human Project Manager (Sole Operator / Architect)	Requirements definition, editorial direction, creative acceptance, platform authentication, and final release authorization.	Explicit authorization events (`USER_AUTHORIZED_*`), mobile/browser verification receipts, and plan confirmation hashes.	Retains exclusive sign-off on public releases, deletions, and schedule overrides; resolves `UNKNOWN` platform states.
GPT / Codex Workflow (Production & Backend QA)	Topic/merchant research, script synthesis, programmable video timeline assembly, FFmpeg encoding checks, and backend repository review.	SHA-256 verified `.mp4` exports, frame-sampling logs, decode error counts, and pull request verification records.	Cannot self-authorize public platform writes; unapproved or in-progress renders (`RENDER_OR_QA_PENDING`) are excluded from release.
Claude Workflow (Interface & Tool Construction)	CLI interface construction, layout adaptation, portrait cut tooling, and developer user-experience refinements in *CrossCast*.	Interface and first-run changes in scoped pull requests (e.g., PRs 8–10) with unit tests; commit metadata does not record which model authored a change.	Changes remain commit-scoped until verified by regression suites; local mock tests are never reported as live speedups.
DeepSeek Workflow (Scheduled Operations Monitor)	Hourly automated heartbeat inspections, reading operational checklists, checking due times, and synchronizing user instructions.	Machine-readable execution traces (`scheduled-run-evidence.json`) and de-identified event logs (`operations-events.json`).	Strictly read-only when no authorized task is due; forbidden from launching duplicate runs or retrying uncertain submissions.

Model names denote workflow roles: the records do not authenticate the model version behind each request (`docs/model-workflow_EN.md`), and the 3 November record describes the person as setting requirements and acceptance, Claude as supporting construction and interfaces, and Codex as leading backend review and final repository updates.

4.2 Layer 1: Deterministic Sole-Operator and Temporal Scheduling Gate

To prevent race conditions and unauthorized autonomous execution during scheduled heartbeats, Layer 1 enforces a pure offline decision function G_{plan} before any network or platform adapter is invoked. Let $S = (e, o, f_{\text{stop}}, f_{\text{run}}, t_{\text{now}}, J)$ denote the operational context tuple, where e is the requesting executor identifier, o is the registered instance owner, $f_{\text{stop}} \in \{\text{true}, \text{false}\}$ is the emergency halt flag, $f_{\text{run}} \in \{\text{true}, \text{false}\}$ indicates whether an execution lock is active, $t_{\text{now}} \in \mathbb{R}^+$ is the current epoch timestamp, and $J = \{j_1, \ldots, j_m\}$ is the set of candidate jobs.

$$G_{\text{plan}}(S) = [\ (\texttt{skip}, \texttt{not_owner}, \emptyset) \text{ if } e = \emptyset \vee e \neq o;\quad (\texttt{skip}, \texttt{stop_flag}, \emptyset) \text{ if } f_{\text{stop}} = \text{true};\quad (\texttt{skip}, \texttt{already_running}, \emptyset) \text{ if } f_{\text{run}} = \text{true};\quad (\texttt{plan_only}, \texttt{checked}, D) \text{ if } D \neq \emptyset;\quad (\texttt{no_work_due}, \texttt{checked}, \emptyset) \text{ otherwise }] \tag{1}$$

where the due set D is strictly filtered as:

$$D = \{\, id_k \mid j_k \in J \wedge done_k = \text{false} \wedge auth_k = \text{true} \wedge t_{due,k} \leq t_{now} \,\} \tag{2}$$

Furthermore, for any automated run record $R = (\text{trig}, \text{status}, f_{\text{manual}}, t_{\text{start}}, t_{\text{end}}, E_{\text{err}})$, post-execution verification $V_{\text{sched}}(R)$ holds if and only if:

$$V_{\text{sched}}(R) \Leftrightarrow (\text{trig} = \texttt{automation_heartbeat_scheduled}) \wedge (\text{status} = \texttt{VERIFIED_AUTOMATIC_RUN_COMPLETED}) \wedge (f_{\text{manual}} = \text{false}) \wedge (t_{\text{start}}, t_{\text{end}} \text{ finite} \wedge t_{\text{start}} \leq t_{\text{end}}) \wedge (|E_{\text{err}}| = 0) \tag{3}$$

Algorithm 1: Deterministic Sole-Operator & Temporal Scheduling Gate (`planRun` & `verifyScheduledRun`)

Input: Context tuple $S = (e, o, f_{stop}, f_{run}, t_{now}, J)$; Run trace record R
Output: Gate decision tuple (decision, reason, D) and boolean verification result v

1: $t_{cur} \leftarrow \text{ParseEpoch}(t_{now})$
2: **if** $\neg\text{IsFinite}(t_{cur})$ **then throw** `Error("Invalid current time")`
3: **if** $e = \emptyset \vee e \neq o$ **then return** `("skip", "not_owner", ∅)`
4: **if** $f_{stop} = \text{true}$ **then return** `("skip", "stop_flag", ∅)`
5: **if** $f_{run} = \text{true}$ **then return** `("skip", "already_running", ∅)`
6: $I_{seen} \leftarrow \emptyset$; $D \leftarrow []$
7: **for each** job $j_k = (\text{id}_k, t_{due,k}, \text{auth}_k, \text{done}_k) \in J$ **do**
8: **if** $\text{id}_k = \emptyset \vee \text{id}_k \in I_{seen}$ **then throw** `Error("Missing or duplicate job id")`
9: $I_{seen} \leftarrow I_{seen} \cup \{\text{id}_k\}$
10: **if** $\text{done}_k = \text{true} \vee \text{auth}_k \neq \text{true}$ **then continue**
11: $t_{at} \leftarrow \text{ParseEpoch}(t_{due,k})$
12: **if** $\neg\text{IsFinite}(t_{at})$ **then throw** `Error("Invalid due time for " + id_k)`
13: **if** $t_{at} \leq t_{cur}$ **then** $\text{Append}(D, \text{id}_k)$
14: decision ← $(|D| > 0)$? `"plan_only"` : `"no_work_due"`
15: **return** (decision, `"checked"`, D)

As formalized in Algorithm 1, `planRun` is a side-effect-free admission filter. It first checks that the reference clock parses to a finite epoch value (lines 1–2), then applies three ordered circuit breakers: executor ownership (line 3), the stop flag (line 4) and the running lock (line 5). Only when all three pass does it scan the candidate jobs, rejecting missing or repeated identifiers (lines 6–9) and skipping jobs that are completed or lack explicit human authorization (line 10). Even when due jobs exist, it returns `plan_only` rather than acting, leaving content checks and confirmation to the later layers. Two properties of the implementation (`scripts/operations-gate.mjs`) matter for interpretation. A non-finite due time raises an error only for jobs that are authorized and not completed, because other jobs are skipped before their due time is parsed. And `planRun` is advisory: it returns a decision but cannot stop an agent that never calls it. In the laboratory the operations checklist requires the call; enforcement at the platform boundary comes from Layers 2 and 3, which sit inside the release command itself.

4.3 Layer 2: Plan-Hash Binding and Artifact Verification

Between plan approval and execution, an agent or background process may change a video export, swap a cover or alter platform copy after the project manager has reviewed the preview. Large deliverables (2.6 MB to 285.9 MB in our archive) can also be written incompletely or misread; the release record of the v3 film notes one earlier local read discrepancy that remains unexplained. Layer 2 closes this time-of-check-to-time-of-use gap with two invariants:

Governance Invariant 1 (Media Integrity): Let $M = \{m_1, \ldots, m_K\}$ be the release manifest, where each entry $m_i = (\text{fname}_i, B_i, H_i)$ gives a file name, an exact byte length B_i and an expected SHA-256 digest H_i. An artifact f_i passes the delivery check iff $\text{basename}(\text{fname}_i) = \text{fname}_i \wedge \text{size}(f_i) = B_i \wedge \text{SHA256}(f_i) = H_i$. Decode and frame checks are recorded separately in production receipts.

Governance Invariant 2 (Plan-Hash Confirmation Binding): Before any platform write through `vpub publish`, the release plan P —target platforms, account identities, video and cover SHA-256 digests, aspect ratio, visibility and platform copy—is serialized as JSON with sorted keys and compact separators and hashed, $H(P) = \text{SHA256}(\text{CanonicalJSON}(P))$. Execution is refused unless the command supplies `--confirm` H_{user} with $H_{user} = H(P)$; a changed plan has a new digest, so a stale code is refused.

Both `check-delivery.mjs` (Node.js) and `vpub` (Python) hash media in streamed chunks rather than loading whole files, and the delivery check rejects manifest names that are not plain file names. The manifest records a zero-error full decode for five of the 30 deliveries (entries 26–30); update notes record decode checks for several others. The plan digest also names the receipt file—one per plan and platform—so a second attempt at the same plan finds the first attempt's receipt instead of starting afresh (Section 4.4). Invariant 2 binds the command to the exact content that was reviewed; it does not authenticate who approved it. The CLI records `--person-approved` as the caller's attestation and states that it cannot verify who supplied it, so human authority rests on procedure (the agent must stop and show the plan) while the digest guarantees that what runs is what was shown. Loudness, clipping, silence-window and frame-scan measurements that accompany deliveries are reported per case in Table 8.

4.4 Layer 3: Idempotent Receipt State Machine and UNKNOWN-State Suppression

To answer **RQ2**, we model each browser publication attempt in *PublishOps CrossCast* as a durable receipt whose changes are written atomically under a file lock, while cross-process OS locks (`process_lock.py`) keep two runs from driving the same browser profile (Figure 2, Algorithm 2). The receipt is stored at a path named by the plan digest and the platform and holds R_p = (key, acct_p, H_{media}, H_{meta}, f_{submit}, σ_p, wid_p, τ_{log}), where f_{submit} is the implementation's `final_submit_attempted` field and σ_p its state. Before pressing, the receipt also records the account's existing work identifiers, so that read-back can later look for exactly one new work. Figure 2 shows the states used at commit `75b7abc` (`browser_backends.py`).

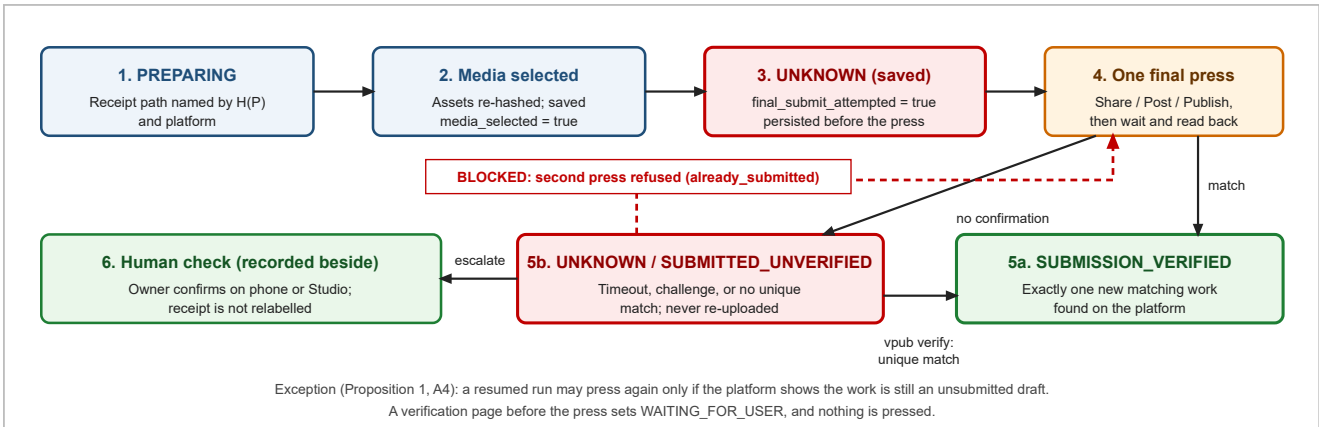


Figure 2. Receipt states of a browser publication attempt in *PublishOps CrossCast* (commit `75b7abc`). The receipt is saved as `UNKNOWN` with `final_submit_attempted = true` before the single final press; a later run refuses a second press unless the platform shows the work is still a draft, and only read-back or a person settles the outcome.

Algorithm 2: Idempotent Cross-Platform Publication & `UNKNOWN`-State Latching (`vpub publish` & `vpub verify`)

Input: Release plan P with digest $H(P)$, confirmation code H_{user}, platform p, account acct_p, resume flag
Output: Persistent receipt R_p = (key, acct_p, H_{media}, H_{meta}, f_{submit}, σ_p, wid_p, τ_{log})

if $H_{\text{user}} \neq H(P)$ **then abort** (stale or wrong plan code)
AcquireProcessLock(p, acct_p) // a stale lock is taken over only after its owner process has exited
path ← ReceiptPath($H(P)$, p)
if Exists(path) $\wedge$ ¬resume **then abort** `attempt_exists` // never reset an attempt to upload again
if resume $\wedge$ Exists(path) **then** R_p ← Load(path); require same H_{media} and acct_p, else **abort** `receipt_conflict`
if resume $\wedge$ ¬Exists(path) $\wedge$ the publication record shows media selected or a press attempted **then abort** `receipt_missing`
VerifySHA256(P.media, P.covers); select media; R_p.media_selected ← true; PersistAtomically(R_p)
if $R_p.f_{\text{submit}}$ = true $\wedge$ ¬PlatformShowsUnsubmittedDraft() **then abort** `already_submitted`
$R_p.\sigma_p$ ← `UNKNOWN`; $R_p.f_{\text{submit}}$ ← true; PersistAtomically(R_p) // before the click
PressFinalSubmitOnce() // a single Share / Post / Publish press
outcome ← AwaitConfirmationAndReadBack(acct_p, new works not listed before the press)
if outcome holds exactly one matching new work **then** $R_p.\sigma_p$ ← `SUBMISSION_VERIFIED`; $R_p.\text{wid}_p$ ← outcome.work_id
else keep $R_p.\sigma_p \in$ {`UNKNOWN`, `SUBMITTED_UNVERIFIED`} // never re-press; escalate to `vpub verify` or a person
PersistAtomically(R_p); ReleaseProcessLock(p, acct_p); **return** R_p

Proposition 1 (At-Most-Once Automated Final Submission). Assume (A1) every final submission for plan P on platform p goes through the receipt transaction of Algorithm 2; (A2) receipt writes are atomic and durable; and (A3) receipts are not edited by hand. Then automated runs issue at most one final press per (P, p), with one exception: (A4) a resumed run may press again when the platform shows that the earlier work is still an unsubmitted draft.

Proof sketch. The receipt is saved with σ_p = `UNKNOWN` and f_{submit} = true before the press (line 9). A later run on the same plan either finds the receipt and stops (line 4) or resumes it, in which case line 8 refuses another press while f_{submit} = true unless (A4) holds. If the receipt file has disappeared while the publication record shows a selected file or an attempted press, the run stops (line 6) instead of uploading again. ■

Proposition 1 is a safety guarantee, not a liveness one. A crash between saving the receipt and pressing leaves an `UNKNOWN` receipt for a submission that never happened; the design accepts such false alarms because the opposite error, a duplicate public post, cannot be undone. Whether an `UNKNOWN` submission published is settled afterwards by `vpub verify`, which checks that the signed-in account is the receipt's account and upgrades the receipt only when it finds exactly one matching new work, or by a person whose confirmation is recorded beside the receipt without relabelling it. Resumable byte offsets belong to API routes (`upload_resume.py` in the laboratory's 0.15.1 snapshot); the browser route instead refuses to upload again once its record shows that a file was selected.

4.5 Layer 4: Identity-Bound In-Place Revision and Consent-Bound Deletion

Beyond first publication, channel operations revise captions and covers, assign chapters and occasionally delete test works. An agent that implements an edit as delete-and-repost, or deletes without confirming the target's identity, turns a reversible correction into an irreversible loss. Layer 4 binds every revision and deletion to the receipt's account and work identifier (Algorithm 3), developed across PRs #20, #21, #25, #29 and #30.

Algorithm 3: Identity-Bound In-Place Revision & Consent-Bound Single-Work Deletion (`vpub edit` & `vpub delete`)

Input: Receipt key K, patch ΔP or deletion request, mode $\in$ {`VERIFY_ONLY`, `APPLY_EDIT`, `DELETE_WORK`}
Output: Edit or deletion state and audit entry

1: $R_p \leftarrow$ LoadReceipt(K); require the signed-in account = R_p.acct$_p$ (`expected_identity`)
2: **if** mode $\in$ {`VERIFY_ONLY`, `APPLY_EDIT`} **then**
3: live $\leftarrow$ ReadNativeEditorFields(R_p.wid$_p$) // page translations and comments are ignored
4: **if** live = ΔP.target **then return** `EDIT_VERIFIED` // nothing to save
5: **if** mode = `VERIFY_ONLY` **then return** `EDIT_MISMATCH`
6: **if** the platform offers no in-place edit for this work **then return** `EDIT_UNAVAILABLE` // never delete or repost instead
7: checkpoint `EDIT_PREPARING`; ApplyInPlace(R_p.wid$_p$, ΔP) // video bytes and visibility unchanged
8: reopen the editor; **return** `EDIT_VERIFIED` if read-back equals ΔP.target, else `EDIT_SUBMITTED_UNVERIFIED`
9: **else if** mode = `DELETE_WORK` **then**
10: require a dry run and a person's approval of this exact work (`--confirm` code, `--person-approved`), stored durably
11: wait until the deletion control is ready; DeleteSingleWork(R_p.wid$_p$)
12: refresh the catalogue; report success only if the work is absent

Algorithm 3 adds three safeguards. First, every read or change begins by confirming that the active session belongs to the receipt's account (`expected_identity`), preventing cross-account changes. Second, an edit compares native editor fields rather than the rendered page, and where the platform offers no in-place edit—as for an X post whose menu has no Edit action—the run ends with `EDIT_UNAVAILABLE` and attempts no deletion, repost or reply: compensation by repost would lose the post's identifier, replies and reach (Section 2.4). Third, a deletion requires a dry run and a person's approval of that exact work, waits until the deletion control is ready (PR #30), and counts as done only when a refreshed catalogue no longer lists the work (PR #29).

5. System Implementation: Agentic Work Lab and PublishOps CrossCast

5.1 Programmable Video Synthesis and Multi-Modal Quality Gates

Within *Agentic Work Lab*, video production is engineered as an editable, code-driven workflow (`editable-video-workflow.md`) rather than opaque one-shot video generation. Audio narration timelines, character pose transitions, camera movements, and bilingual subtitles are defined programmatically so that localized defects can be repaired in place without regenerating unaffected segments (Figure 3).

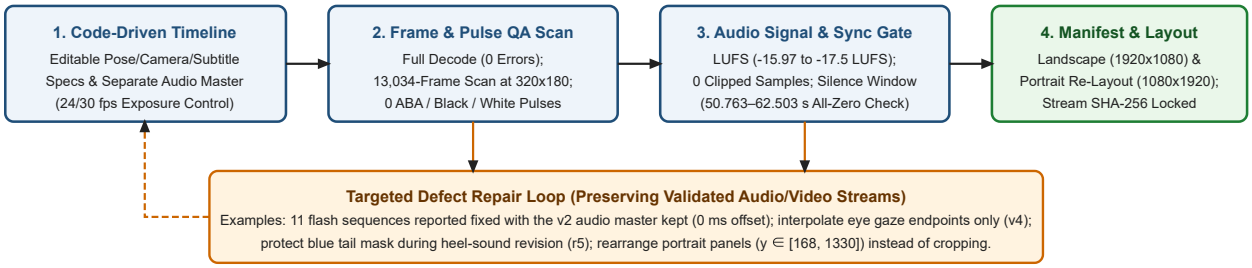


Figure 3. The programmable video synthesis and multi-modal quality verification pipeline in *Agentic Work Lab* (values from the delivery records summarized in Table 8).

5.2 CrossCast Two-Slot Concurrent Browser Architecture and Session Reuse

PublishOps CrossCast was released publicly as `v0.16.0` (commit `a46bd19`) and developed on its main branch to commit `75b7abc` (PR #30, 3 November 2026), the snapshot evaluated here. At that commit the repository holds 123 Python source files (26,972 physical lines): 81 files (18,005 lines) form *CrossCast*'s own package, and 42 files (8,967 lines) are an unmodified copy of the MIT-licensed `bili-cli` v1.2.1, bundled for Bilibili access (`NOTICE.md`). Its 126 test files (122 test modules and four shared helpers; 37,029 lines) define 2,737 test functions, up from 2,009 in 84 files at `v0.16.0`, and GitHub's continuous-integration checks for `75b7abc` passed on Python 3.10–3.14. Because one Instagram run spent most of its time waiting after submission (Table 7), PRs #11 and #15 (`066c49f`, `6f7b62b`) added a bounded two-slot scheduler for browser platforms (Figure 4): each platform keeps its own account lock, browser session and receipt, up to two browser platforms publish at once, and the next queued platform starts when a slot frees. No concurrent publication has yet been timed, so no speedup is claimed. API routes for YouTube, X and Instagram are distributed separately from the public repository and lie outside the evaluated snapshot.

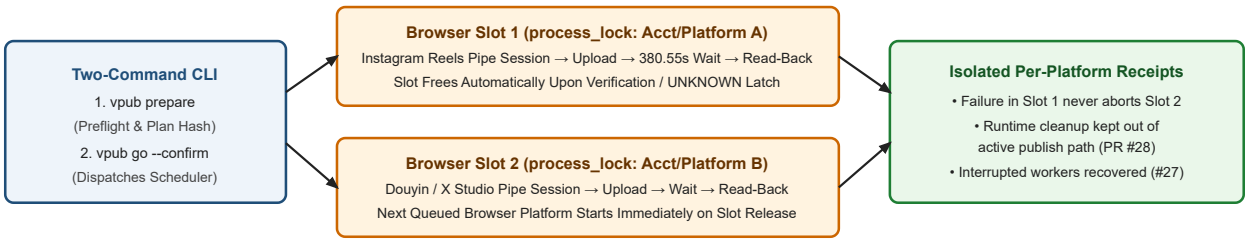


Figure 4. Two-command release workflow (`prepare` / `go`) and bounded two-slot concurrent browser publication in *PublishOps CrossCast* (PRs #10, #11, #15, #27, #28). Architecture only: the Instagram wait shown in Slot 1 comes from a sequential run, and no concurrent run has been timed.

6. Empirical Evaluation and Results

6.1 Operational Journal and Write Evidence (RQ1)

The operational journal (`data/operations-events.json`) is an export of selected fields from the laboratory's local event logs. It holds 69 records, of which 46 are timestamped, between 26 October 01:27 and 27 October 02:45 (UTC+8, a 25.3-hour window); it was exported on 28 October. Because the export kept different fields for different records—33 carry an event name, 5 only a state

name and 31 neither—we classify the records by the write evidence each one carries rather than by a coding of their purpose (Table 3).

Table 3: Write evidence in the *N* = 69 records of the operational journal, classified by the fields each record carries (recomputed by `reproduce_paper_tables.mjs`).

Evidence Carried by the Record	Records	Share	Platform Reads	Platform Writes	What the Record Shows
Explicit zero-write field (`platform_writes`, `write_calls` or `platform_write_calls` = 0)	20	28.99%	13	0	States that no write occurred; includes the four read-only sampling runs (26 CLI calls) and the four automation-triggered checks.
Zero platform calls (`platform_calls` = 0; no write field)	39	56.52%	0	0	No platform call, hence no write.
One platform call, no write field (record 22)	1	1.45%	—	—	Event name indicates a sign-in check after a title choice; reads and writes not recorded.
One platform write (record 52)	1	1.45%	4	1	Unnamed in the export; four reads recorded with it; `new_risk` = false.
No call or write field	8	11.59%	—	—	Silent on writes; the absence of a write is not shown.
Total	**69**	**100.00%**	**17**	**1**	**No record reports a replayed upload.**

Table 3 supports three statements and rules out a fourth. (i) One record reports a platform write (record 52: one write and four reads, `new_risk` = false); the export does not name the action. (ii) No record reports a replayed upload: the replay counter appears in one record (value 0), and all eight records that carry `new_risk` report false. (iii) 59 records carry an explicit zero count of writes or platform calls; the other nine are silent on writes. What the journal cannot show is that a gate blocked an attempted write: it records outcomes, not refused attempts, so the absence of writes in 68 records is consistent with, but not evidence of, suppression. Evidence that Layer 1 makes the intended decisions comes instead from the four automation-triggered records (all without writes and with the stop flag off), the heartbeat trace below, and the workflow tests (10/10 passed, `results/validation.json`).

The audited DeepSeek heartbeat (`scheduled-run-evidence.json`, planned for `2026-10-28T12:37:13.943+08:00`) started after a delay of 24,992 ms (24.992 s) and completed its four recorded work items—reading ownership, checklist, stop conditions and checkpoints; finding no comment, component, research or analytics work due; synchronizing instructions; and needing no Bilibili operation—in 31,175 ms (31.175 s), with no task errors, no manual test trigger and no duplicate run. The record satisfies V_{sched} (Eq. 3).

6.2 Software Verification and Design Science Evolution (RQ1 & RQ2)

Live experiments must not pollute public accounts with deliberate faults such as double posting or deleting production videos, so offline tests are the main evidence for edge-case transitions; one live crash-recovery check was run on a private clip (Table 5). Table 4 summarizes the *CrossCast* test suite at commit `75b7abc`, grouped by file name, and Table 5 traces the design iterations. Test-function counts measure the volume of checking, not coverage; no coverage tool was run.

Table 4: Code and offline test volume of *PublishOps CrossCast* at commit `75b7abc`, with test files grouped by file name (first matching rule; a file may exercise several concerns), and the *Agentic Work Lab* entry points. Lines are physical lines.

Group / Entry Point	Files	Lines	Test Functions	Examples of Behaviour Exercised
A. PublishOps CrossCast at `75b7abc` (123 source files, 26,972 lines, of which 42 files are bundled `bili-cli v1.2.1`)				
1. Publishing workflow, receipts, locks & recovery	20	7,547	494	Two-command `prepare` / `go` flow, publish-plan safety, worker slots and refill, checkpoint de-duplication, browser broker and lock lifecycle.
2. Deletion & absence read-back	19	6,830	340	Consent, dialogs, loading, timelines and absence read-back for single-work deletion across platforms.
3. Edits & metadata read-back	5	1,615	128	X edit consistency and length, Bilibili mention links, staged edits.
4. Platform pages & adapters	20	7,154	590	YouTube Studio, X site and schedules, Douyin forms, Instagram media and comments.
5. Collections & playlists	8	2,792	259	Collection membership, page redraws, empty and incomplete playlists.
6. Messages, comments, engagement & safety	18	4,300	402	Comment snapshots, replies, likes, hostile-message flags and rate limits.
7. Other CLI, documentation & installation	32	6,383	524	Capabilities, first run, preferences, covers, installer and documentation checks.
Subtotal (incl. 4 shared helper files, 408 lines)	**126**	**37,029**	**2,737**	**Test-to-source line ratio 1.37 (2.06 against *CrossCast*'s own 18,005-line package).**
B. Agentic Work Lab Entry Points (Node.js v26.11.1)				
8. Workflow gate & merchant demo (`workflows.test.mjs`)	1	27	10	Non-owner rejection, stop flag, running lock, invalid due time, scheduled-run verification (10/10 passed; `results/validation.json`, 28 Oct).
9. Archive integrity record (`media/manifest.json`)	1	—	30 records	Size and SHA-256 of 30 GitHub assets recorded as verified on 3 Nov; `SHA256SUMS.txt` matches the manifest.

Table 5: Design Science evolution of *PublishOps CrossCast* from early prototypes to commit `75b7abc` (PR #30).

Version / Commit	Architectural Milestone	Empirical & CI Verification Evidence	Identified Boundary & Subsequent DSR Refinement
`v0.2.2–v0.3.0` (lab commit `596e951`)	Initial 5-platform adapters, X Media Studio 16:9 thumbnail persistence, FFmpeg 79.5s cover-lead synthesis.	86 offline tests passed; verified YouTube Studio category change & 1,327-char Instagram caption read-back.	Instagram profile grid cropped landscape cover text; TikTok suspended after a sign-in with a Hong Kong (+852) number failed.
`v0.15.1` (Lab Snapshot)	OS process lock (`process_lock.py`), private pipe browser context reuse, resumable byte-offset recovery.	417 offline tests passed; a private YouTube upload forced to exit after 8,388,608 acknowledged bytes resumed in a second process and kept the same video ID (30 Oct).	Needed independent CI matrix verification and stricter schedule/pin counterexample checks.
Candidates `eaa1aab` / `80e166b` / `2d54c8f`	Separated development from independent validation; tested X schedule submission and cleanup.	GitHub CI passed on Python 3.10–3.12; 143 + 16 targeted local checks passed; X temp media deleted & confirmed absent.	Counterexample found imprecise pinned-comment ID (live pin held); X schedule row lacked post/media ID pair (kept `SUBMITTED_UNVERIFIED`).
`v0.16.0` Release (`a46bd19` / PR #8–#9)	Public standalone CLI release; shortened default capability list (`--full`) and separated rehearsal from live readiness.	Suite of 2,009 test functions in 84 files at `a46bd19`. Field release of *I Watched It For You (v3)* on 2–3 Nov with one instrumented Instagram run (412.217 s); the release record does not name the code version used per platform.	Post-submit wait consumed 92.32% of publish time; motivated explicit wait progress and bounded concurrency.
PRs #10–#28 (`d97835f`–`439c038`)	Two-command `prepare`/`go` flow (#10), two-slot browser concurrency (#11/#15), YouTube single-Save guard (#16), X edit consistency (#21).	Python 3.13/3.14 support (#14); prompt login return & interrupted worker recovery (#27); runtime cleanup isolated (#28).	Required durable consent and post-deletion absence read-back for single-work deletions across all platforms.
PRs #29–#30 (`3b56bdc` / `75b7abc`)	Durable-consent single-work deletion with absence read-back (#29); deletion control readiness & layout guards (#30).	Suite at **126 test files (37,029 lines, 2,737 test functions)**; GitHub CI checks passed on Python 3.10–3.14.	Live multi-platform concurrent speedup benchmarking scheduled as next empirical evaluation phase.

6.3 Cross-Platform Release Verification and Latency Bottleneck Analysis (RQ2 & RQ3)

To examine the release layers under real platform conditions, we use the release of the 176.4 s short film *I Watched It For You (v3)* (`i-watched-it-for-you-v3-20261102.mp4`, 285,855,005 bytes, SHA-256 `7d6fd76090f7a93b2711e15db4bb924f741d3a904a01fc4b7628fc2130dbafdc`) on five platforms on 2–3 November 2026 (Table 6), and the phase timing recorded for its Instagram publication (Table 7, Figure 5). Each platform entry comes from its own publication run or from the owner's confirmation; the 3 November record made no fresh platform queries.

Table 6: Cross-platform publication record for *I Watched It For You (v3)* (`production-update-2026-11-03.json`). States are quoted as recorded; "—" means the record holds no such field.

Platform	Route or Continuity (as Recorded)	Native Receipt State	Recorded Verification State	Outcome and Evidence
Bilibili	Production-session report; not re-queried (`fresh_check` = false)	—	`public_reported_by_production_session` (`BV15B4F6hEYr`)	Public availability recorded on 2 Nov at 21:01 UTC+8.
YouTube	Continued from the original draft	`SUBMISSION_VERIFIED`	`PUBLIC_VERIFIED` (`AzW6vr8nvDM`)	Native receipt, full copy and public visibility passed read-back.
X	Reused the original uploaded media	`SUBMISSION_VERIFIED`	`PUBLIC_VERIFIED` (`2117261340633161838`)	Receipt and 176.4 s duration verified; stored resolution not established.
Douyin	Programmatic read-back timed out	**`UNKNOWN`**	`USER_CONFIRMED_VISIBLE_NATIVE_VERIFY_BLOCKED`	Neither relabelled nor republished; the owner's phone screenshot shows the work and production cover.
Instagram	One upload and one Share (3 Nov)	`SUBMISSION_VERIFIED`	— (`metadata_readback` = true)	Full Chinese/English caption read back; original ratio; AI label on; cover replaced by the owner on mobile.

The Douyin entry is the field instance of Proposition 1. The run pressed Publish once; its programmatic read-back then timed out, and the receipt kept the `UNKNOWN` state it had been given before the press. No run submitted the 285.9 MB file again, and the owner's phone confirmed that the work and its production cover were live—a confirmation recorded beside the receipt rather than written into it. A separate test on 1 November had the same shape: a Douyin scheduled card showed the requested time and private marker, the CLI read-back timed out, and the failure receipt was kept as recorded while the temporary item was removed. These are two instances, not a rate. The records do not say why read-back timed out, and the claim that a retrying agent would have published a duplicate is an inference from the retry policy (Table 11), not an observation. The Instagram row shows a second, benign use of the human channel: the owner replaced the cover on mobile, and the record notes that remote cover pixels were not verified.

Table 7: Phase timing of one Instagram publication ($N = 1$; `production-update-2026-11-03.json`). The source selects major phases rather than an exhaustive additive breakdown; row 7 is derived.

Execution Phase	Execution Scope	Duration (s)	% of Cmd	% of E2E
1. Source & Release Plan Cryptographic Checks	Outside Publish Cmd	0.438	—	0.10%
2. Local Dry-Run Rehearsal	Outside Publish Cmd	0.444	—	0.10%
3. Sign-In Preflight & Session Close	Outside Publish Cmd	10.137	—	2.40%
Subtotal: Pre-Publication Governance Checks	***Outside Publish Cmd***	***11.019***	***—***	***2.60%***
4. Browser Start to Media File Entry	Inside `vpub publish`	11.058	2.68%	2.61%
5. Wait after the single Share (transfer, processing & confirmation; not separable)	**Inside `vpub publish`**	**380.551**	**92.32%**	**89.91%**
6. Independent Post-Publication Read-Back	Inside `vpub publish`	5.224	1.27%	1.23%
7. Residual (derived: command total minus rows 4–6)	Inside `vpub publish`	15.384	3.73%	3.63%
Subtotal: Publication Command Total	***Inside `vpub publish`***	***412.217***	***100.00%***	***97.40%***
Total End-to-End Observed Workflow Time	**All Recorded Phases**	**423.236**	**—**	**100.00%**

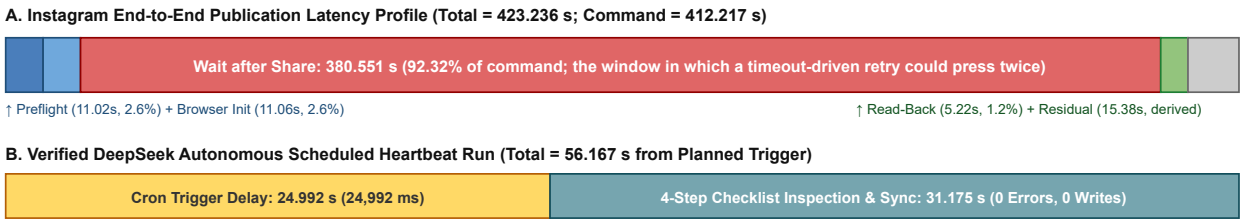


Figure 5. Timelines of (A) the instrumented Instagram publication run ($N = 1$), in which the wait after the single Share made up 92.32% of the publish command and the receipt had been saved before the press, and (B) the verified DeepSeek heartbeat run ($N = 1$).

6.4 Multi-Modal Quality Engineering and Complete 30-Artifact Catalogue (RQ3)

To show how Layer 2 governs deliverable quality across revisions, Table 8 summarizes frame-level and audio measurements recorded in the production studies, and Table 9 lists all $N = 30$ video versions archived in *Agentic Work Lab* with their recorded SHA-256 digests.

Table 8: Programmable video quality cases and targeted repairs, with measurements as recorded in the cited documents.

Production Case	Diagnosed Technical Defect	Surgical Code / Timeline Repair	Quantitative Verification & Documented Limits
Animation Timing Diagnostic (`animation-timing.md`)	Changing render from 24 to 30 fps failed to create missing contact, load-transfer, and withdrawal actions.	Decomposed 24 fps exposure rules (ones = 24, twos = 12, threes = 8 exp/s) and 5-stage physical handoff criteria.	66.7 s (2,001 frames at 30 fps): 52 segments, 44 holds (median hold 0.85 s, max 7.0 s). Established diagnostic baseline.
GPT Duty Log 05 (`v3-antiflash` , 434.47 s)	Full-opacity frame before fade, white flashes during right-panel changes, and shot leakage at cuts.	Set explicit hidden initial states, frame-aligned boundaries, whole-panel replacement, and 4-frame clean-entry hold; kept v2 audio.	**13,034 / 13,034 frames scanned** at 320×180: 11 sequences reported fixed; 0 ABA/black/white pulses; −15.97 LUFS, −2.21 dBTP, 0 ms audio offset.
Whale Girl Transformation (`v3` 67s → `v4` 75.23s)	Abrupt eye gaze jumps, brightness mismatch across cut, and moving tail fin entering fixed hair mask.	Interpolated eye regions only between real source endpoints; matched cut brightness; expanded v4 protection mask; ducked music 12 dB.	1080×1920, 2,257 frames: 0 decode errors, 0 clipped samples (sample peak ≈ −1.636 dBFS), all-zero silence verified from 50.763 to 62.503 s, black frame at 1,845 (61.500 s).
Whale Girl Revisions (`r5` Colour/Heels & `r7` Motion)	Colour spill onto blue tail during grading (r5); uneven alternating steps in sock/heart gestures (r7).	Separated clothing/skin/background masks protecting tail & 6 heel events (r5); reordered real source frames without optical flow (r7).	r5: 66.87 s (2,006 frames), full decode passed (+1.11 dBFS inherited float peak noted). r7: 65.37 s (1,961 frames), join & duplicate-frame checks passed.
Whale Girl Ep 07 & Portrait Cuts A/B/C (179.77 s + 3 cuts)	Cropping 1920×1080 landscape to 1080×1920 portrait cuts off ticket counters and character art.	Programmatically rearranged ticket/character panels into $y \in [168, 1330]$ (56 px subtitles, 106 px numbers) at 360 px preview width.	5,393 landscape frames + 843/897/899 portrait frames; portrait cuts −17.5/−16.6/−17.5 LUFS, peaks ≈ −1.5 dBFS, 0 clipped samples. Documented 130 ms first-ticket stamp offset vs. earlier 6 ms estimate.

Table 9: All *N* = 30 downloadable video versions archived in *Agentic Work Lab* (2,211,716,129 bytes; 2.21 GB or 2,109.26 MiB). GitHub asset sizes and SHA-256 digests were recorded as verified on 3 November 2026, and `SHA256SUMS.txt` matches the manifest. State: Bilibili state, current where recorded, otherwise at snapshot.

#	Archive Filename	Deliverable Title / Scope	MiB	Exact Bytes	SHA-256 Prefix	Dur (s)	State
1	`gpt-log-01.mp4`	GPT Duty Log 01	153.01	160,441,478	`13a9b82396e5da…`	—	`published`
2	`gpt-log-02.mp4`	GPT Duty Log 02	164.55	172,545,563	`decb95dbb5ebaa…`	—	`published`
3	`gpt-log-03.mp4`	GPT Duty Log 03	42.38	44,443,753	`b6932ebc4f5821…`	—	`published`
4	`whale-log-01.mp4`	Whale Girl Shift Log 01	40.21	42,158,416	`5fc2a8bc961a49…`	—	`published`
5	`whale-log-02.mp4`	Whale Girl Shift Log 02	33.70	35,336,986	`6bd55d5a62026f…`	—	`published`
6	`whale-log-03.mp4`	Whale Girl Shift Log 03	23.84	24,994,028	`95a862fd0e9ed8…`	—	`published`
7	`gpt-log-04.mp4`	GPT Duty Log 04: production tutorial	157.70	165,360,386	`870fd0dd15459a…`	—	`published`
8	`gpt-log-05-review.mp4`	GPT Episode 05: Seedance beginner tutorial (review copy)	105.48	110,600,280	`2e4a5da549b341…`	—	`ready_for_review`
9	`archive-episode03-candidate.mp4`	Earlier local episode 03 candidate (not the released episode)	169.51	177,747,769	`3c2225790f5f17…`	—	`unpublished_experiment`
10	`archive-episode04-story-v4.mp4`	Earlier local episode 04 story cut (not the released episode)	67.51	70,793,338	`96da49a29f44ae…`	—	`unpublished_experiment`
11	`archive-triangle-cats.mp4`	Triangle-cats challenge candidate	72.21	75,712,985	`7c081526da7a24…`	—	`unpublished_experiment`

#	Archive Filename	Deliverable Title / Scope	MiB	Exact Bytes	SHA-256 Prefix	Dur (s)	State
12	`archive-dinner-full-film.mp4`	Earlier complete dinner-animation experiment	130.34	136,670,959	`6b1927cc279e2e…`	—	`unpublished_experiment`
13	`study-connected-motion-v6.mp4`	Ten-second continuity experiment v6	2.44	2,562,181	`41a2929321d5f3…`	—	`research_sample`
14	`study-comic-v3-1.mp4`	Comic-style short experiment v3.1	13.78	14,449,925	`adef504edb059f…`	—	`research_sample`
15	`study-3d-handover-v2.mp4`	3D walk-and-handoff prototype v2	23.56	24,704,815	`b409355012deff…`	—	`research_sample`
16	`study-seedance-previs.mp4`	Seedance production previs	6.20	6,500,102	`33132eddbeb8a1…`	—	`research_sample`
17	`whale-log-04-v2-20261028.mp4`	Whale Girl Shift Log 04: small favours	11.93	12,506,936	`c23c171149bc47…`	132.00	`scheduled_on_bilibili`
18	`whale-log-05-v2-20261028.mp4`	Whale Girl Shift Log 05: the bill	13.40	14,052,172	`f735aca1bb770a…`	148.20	`scheduled_on_bilibili`
19	`gpt-log-05-v3-antiflash-20261028.mp4`	GPT Duty Log 05: Seedance tutorial (flash correction)	83.52	87,578,698	`5dc121e833dcfc…`	434.47	`scheduled_on_bilibili`
20	`gpt-log-06-v2-20261029.mp4`	GPT Duty Log 06: Step by step, getting DeepSeek to help with my work (v2)	42.23	44,282,436	`b53d3df6f85b2e…`	591.47	`scheduled_on_bilibili`
21	`whale-log-06-v2-20261029.mp4`	Whale Girl Shift Log 06: AI does the automation; I pretend to be human (v2)	13.88	14,552,237	`35bc463bc4c7fc…`	184.20	`scheduled_on_bilibili`
22	`study-whale-transformation-v3-20261030.mp4`	Whale Girl transformation study v3 (earlier delivery)	85.84	90,012,465	`60f03a12439c0c…`	67.00	`unpublished_experiment`
23	`study-whale-transformation-v4-20261030.mp4`	Whale Girl transformation study v4 (targeted repair)	98.33	103,106,107	`3b209af916b128…`	75.23	`unpublished_experiment`
24	`study-whale-color-heels-r5-20261031.mp4`	Whale Girl study: local colour and heel-sound revision 5	108.10	113,352,299	`19ea894908ce36…`	66.87	`not_verified`
25	`study-whale-motion-r7-20261101.mp4`	Whale Girl study: motion-timing revision 7	103.48	108,503,225	`933e1643a73519…`	65.37	`not_verified`
26	`whale-log-07-v3-20261101.mp4`	Whale Girl Shift Log 07: duty simulator (v3)	47.02	49,307,630	`e37285ee4f8c9f…`	179.77	`not_refreshed`
27	`whale-log-07-cut-a-v3-20261101.mp4`	Episode 07 portrait cut A: running out of resources	7.50	7,861,825	`8429157c4fe444…`	28.10	`not_refreshed`
28	`whale-log-07-cut-b-v3-20261101.mp4`	Episode 07 portrait cut B: refusing every request	7.81	8,185,740	`1e436aff188953…`	29.90	`not_refreshed`
29	`whale-log-07-cut-c-v3-20261101.mp4`	Episode 07 portrait cut C: bluffing through the shift	7.19	7,536,390	`e7ba91f7e903e2…`	29.97	`not_refreshed`

#	Archive Filename	Deliverable Title / Scope	MiB	Exact Bytes	SHA-256 Prefix	Dur (s)	State
30	`i-watched-it-for-you-v3-20261102.mp4`	I Watched It For You (v3)	272.61	285,855,005	`7d6fd76090f7a9…`	176.40	`not_refreshed`
30	**Total Verified Archive**	**7 GitHub release tags**	**2,109.26**	**2,211,716,129**	**`0 mismatches`**	—	—

6.5 Descriptive Cohort Analytics and Policy Comparison (RQ3)

Table 10 reports descriptive statistics for all $N = 55$ channel records ($n = 45$ human-led history, $n = 10$ AI-assisted series) and two 24-hour checkpoints. Table 11 then reasons about how two simpler governance policies would have handled twelve scenarios from the project, beside the behaviour the four layers showed.

Table 10: Descriptive statistics of the human-led cohort ($n = 45$ published) and the AI-assisted series ($n = 10$: 5 published, 5 scheduled) in the 28 October 2026 snapshot, with two 24-hour checkpoints (sample SD; linear quartiles).

Cohort / Metric	Mean	SD	Median	IQR (Q1–Q3)	Min–Max	Sum
A. Historical Human-Led Cohort (human_led_history: n = 45 published, Oct 2019 – Oct 2026)						
Lifetime Views	40,884.31	64,604.81	15,407.0	1,722–67,502	274–333,725	1,839,794
Likes	4,022.36	7,281.57	729.0	89–3,765	8–30,299	181,006
Comments	78.18	132.65	14.0	4–113	0–597	3,518
Favorites	1,584.31	2,048.27	473.0	19–2,746	2–6,158	71,294
Coins	356.38	751.32	54.0	6–237	2–3,044	16,037
Shares	764.76	1,594.05	31.0	1–996	0–5,571	34,414
Danmaku (Bullet Chats)	190.27	232.37	105.0	41–279	0–1,045	8,562
B. Current AI-Assisted Series (current_ai_series: n = 5 published, 5 scheduled, 25–28 Oct 2026)						
Early Views (age 1.6 h–3.1 days)	133.20	99.53	183.0	42–199	11–231	666
Likes	10.60	7.70	9.0	8–11	2–23	53
Comments	29.40	46.94	12.0	7–13	2–113	147
Favorites	4.20	5.26	2.0	1–5	0–13	21
C. Age-Matched 24-Hour Post-Publication Checkpoints (Single-Work Descriptive Records)						
GPT Duty Log 03 (`BV1UXxN6aECf` , 70 s, age 24.38 h): 229 views, 8 likes (3.49%), 7 replies (3.06%), 44.64% completion (31 s avg watch).						
Whale Girl 02 (`BV1Wfx56JEih` , 117 s, age 24.05 h): 308 views, 24 likes (7.79%), 51 replies (16.56%), 36.63% completion (42 s avg watch).						

The two cohorts are not comparable as samples of production methods. The 45 human-led works were published between October 2019 and October 2026 and accumulated years of exposure (mean 40,884.31 views, median 15,407.0), whereas the five published AI-series works were 1.6 hours to 3.1 days old at sampling (mean 133.20, median 183.0). Every human-led work has more views than every AI-series work (minimum 274 against maximum 231), so a rank test would only restate the exposure gap; we therefore report no significance test. The cohort label also does not assert that older works used no AI tools (`docs/research-method_EN.md`). Channel-level figures invite the opposite error: weekly plays rose to 76,162 from 21,398 (`channel-weekly.json`), but the platform's top-five list for 27 October was led by two older travel vlogs (215 and 210 plays) with AI-series episodes at 30, 18 and 11 plays, and 88.09% of plays came from TV clients—a pattern that does not support attributing the rise to the new series. The summary script labels its export `descriptive_snapshot_not_causal_model_comparison` , and the age-matched checkpoints (Panel C) remain single-work descriptions: differences in duration (70 s vs. 117 s), persona and comment threads preclude attributing them to the underlying models.

Table 11: Analytical comparison of governance policies across twelve scenarios from the project. Baselines A and B were not executed; their entries state what each policy's rules imply for the recorded situation. The right column reports observed or tested behaviour with its evidence type: F = field record, T = offline test, R = review record, C = code change.

Scenario	Baseline A: Retry on Failure (Reasoned)	Baseline B: Schedule Gate Only, No Receipt Latch (Reasoned)	Proposed Four Layers: Observed or Tested Behaviour
1. Douyin read-back timeout after Publish (3 Nov film)	Would treat the timeout as failure and submit the 285.9 MB file again.	Would retry once the timeout expired.	**F:** receipt stayed `UNKNOWN`; no re-submission; owner confirmed by phone.
2. 380.551 s wait after the Instagram Share	A client timeout could trigger a second Share.	Exposed if the session dropped mid-wait.	**F:** one upload, one Share, no retry; receipt saved before the wait; wait progress shown (#13).
3. Wake-up of a non-owner or older executor	Would act on the checklist concurrently.	Blocked by the owner check.	**T:** `planRun` returns `(skip, not_owner)`; **F:** the earlier instance's schedules were paused; OS lock (`process_lock.py`).
4. Heartbeat while a render or QA is pending	Could publish an unverified render.	Would publish once the due time passed.	**F:** a daily check recorded `RENDER_OR_QA_PENDING` and published nothing; jobs also need `auth` = true.
5. Flash frames in a tutorial render (GPT Ep 05)	Would release the defective render.	A hash check would pass the uninspected render.	**F:** 13,034-frame scan; 11 sequences reported fixed; v2 audio master kept.
6. No native Edit action on an X post	Would delete and repost, losing replies and the post ID.	Would stop without reconciling state.	**C/T:** run ends with `EDIT_UNAVAILABLE`; no deletion, repost or reply is attempted.
7. YouTube Studio Save button re-arming	Could press Save twice.	Save state left unverified.	**C/T:** verification after one final Save without re-arming (PR #16, `0969064`).
8. Imprecise pinned-comment selector (`988bee2`)	Could pin the wrong comment.	No counterexample gate before the live action.	**R:** independent counterexamples found the flaw; live pin actions were held.
9. X scheduled row without a durable post/media ID	Would report a verified public post.	Would over-claim verification.	**F:** receipt kept `SUBMITTED_UNVERIFIED`; schedule and media deleted and confirmed absent.
10. Deleting one test upload	Could delete the wrong work.	No absence check after deletion.	**C/T:** dry run, a person's approval of the exact work, refreshed absence read-back (PRs #29, #30).
11. Worker interrupted mid-login or cleanup	Could leave a stale lock that blocks later runs.	Stale lock needs manual removal.	**C/T:** a lock is taken over only after its owner process has exited; cleanup kept out of publishing (#27, #28).
12. Lifetime views compared across cohorts (45 vs. 5 works)	Would claim AI under-performance, or credit the series with 76k weekly plays.	Would report raw means without age control.	**F:** export labelled `descriptive_snapshot_not_causal_model_comparison`; 24-hour checkpoints.

7. Discussion

7.1 Theoretical Implications for Project Management and Human–AI Governance

The study adds to work on AI in project management (Holzmann et al., 2022; Müller et al., 2024; Raisch & Krakowski, 2021) in four ways.

1. **Stage gates become commit protocols when agents act.** Prior multi-agent frameworks coordinate handoffs mainly through role prompts (Wu et al., 2024; Hong et al., 2024; Qian et al., 2024). In this case the controls that held were those moved out of prompts into code at the point of action: an admission function, a content digest that the command must present, and a receipt written before the irreversible step. Read through Section 2.4, a stage gate in agentic delivery is less a meeting than a commit protocol, with configuration management (Whyte et al., 2016) supplying its notion of identity.
2. **Automation authority is state-dependent.** Levels-of-automation models treat autonomy as a design-time choice (Parasuraman et al., 2000; Shneiderman, 2020). Here autonomy changes with the receipt's state: agents act freely while nothing irreversible has happened and lose the authority to retry as soon as a final submission may have occurred.
3. **At-most-once moves work to people, and that work needs design.** Choosing at-most-once over at-least-once converts possible duplicates into in-doubt cases that a person must resolve, which is Bainbridge's (1983) irony in a new setting: the human receives exactly the cases automation cannot settle. The Douyin case was closed with a phone screenshot; governance designs should budget for this work and give the person a read-back command and an evidence bundle rather than a bare alert.

4. **Epistemic governance is a project deliverable.** Provenance fields, SHA-256 manifests and non-causal labels in exported summaries made it possible to report what happened without claiming more. Where AI adoption is often reported through gains that cannot be checked, this reporting discipline is itself part of governance.

7.2 Answers to the Research Questions

- **RQ1 (governance architecture and handoff control).** Stage-gate principles can be expressed as four software-enforced layers whose decisions are deterministic and testable (Section 4). Evidence: the gate's decision function and its ten passing tests, four automation-triggered records and one audited heartbeat without writes, and a journal recording one write and no replay (Table 3). Limit: the journal records outcomes, so it cannot show blocked attempts, and `planRun` binds only agents that call it.
- **RQ2 (idempotent release engineering and recovery).** A receipt saved as `UNKNOWN` before a single final press yields at-most-once automated submission under assumptions A1–A4 (Proposition 1), with read-back or a person deciding the outcome. Evidence: the code path at `75b7abc`, 2,737 offline test functions including fault paths, a private crash-recovery check that kept one YouTube video ID across two forced exits, and two Douyin read-back timeouts handled without re-submission or relabelling. Limit: few field instances, and safety is paid for with false in-doubt cases.
- **RQ3 (empirical characteristics and boundaries).** Local governance checks were small next to platform waiting: in one Instagram run, pre-publication checks took 11.019 s and the wait after Share 380.551 s (92.32% of the publish command). Quality checks caught frame and audio defects (Table 8), and the reporting protocol kept non-comparable cohorts apart (Table 10). Limit: single observations from one laboratory over nine days (Section 7.4).

7.3 Managerial and Engineering Lessons Learned

- **Measure the Wait Before Optimizing Concurrency:** In the one instrumented run, local governance checks took 11.019 s and the wait after Share 380.551 s (Table 7). Speeding up local steps would change little; safe session isolation and bounded concurrency (Figure 4) are the more promising levers, although their gain has not yet been measured.
- **Design Editable Intermediate Representations:** Decomposing deliverables into code-driven timelines, separate audio tracks and structured JSON patches lets project managers enforce targeted revisions (Table 8) without discarding validated upstream work.
- **Preserve Original State on Verification Interruptions:** When platform security challenges or missing edit permissions occur, tools should never delete, repost or overwrite existing works automatically, and a human confirmation should be recorded beside the receipt rather than overwriting it.
- **Budget for In-Doubt Resolution:** An at-most-once design produces cases that only a person can close; give that person a read-back command and the evidence, and track how long such cases stay open.

7.4 Threats to Validity and Limitations

Following case-study guidance for software engineering (Runeson & Höst, 2009), we report the boundaries of the evidence.

- **Internal validity.** The Instagram timing (412.217 s) and the heartbeat (31.175 s) are single instrumented observations, and client-side timing cannot separate transfer, processing and confirmation within the 380.551 s wait. The operational journal is a selected-fields export: 31 of its 69 records carry no event or state name, nine are silent on writes, and it records outcomes rather than refused attempts. An earlier local read discrepancy on the film's file remains unexplained; later reads matched, which does not show that its cause was removed.
- **Researcher role.** The author built, operated and evaluated the artifacts. We limit the risk of motivated reporting by relying on receipts written when each event happened, publishing all data and code, and recomputing every figure with a script; the study nonetheless remains a single-researcher intervention (Sein et al., 2011; Wieringa, 2014).
- **Construct validity.** Suppression of writes is not directly measurable from outcome logs (Section 6.1). Model names denote workflow roles, not authenticated model versions. Test-function counts measure the volume of checking, not coverage. Plan-hash confirmation binds content, while human approval is attested by the caller. The blinded identical-task comparison of GPT and DeepSeek (`docs/model-workflow_EN.md`) has not been run, so no claim is made about relative model capability, speed or cost.
- **External validity.** Evidence comes from one laboratory, one account per platform and a nine-day window, with TikTok suspended after a sign-in with a Hong Kong (+852) number failed. Platforms differ structurally—browser versus API routes, and human confirmation on Douyin—so the latency figures do not transfer. What we expect to transfer is the mechanism: any

agent that submits to an endpoint without server-side idempotency keys faces the same retry dilemma. Generalization is therefore analytical, by architectural similarity, rather than statistical (Flyvbjerg, 2006; Wieringa & Daneva, 2015).

- **Reliability.** The repositories have changed since the snapshot (later *CrossCast* pull requests add tests, for example); all figures refer to the files and commits named in Section 3.3. Line counts are physical lines, and the descriptive statistics use sample standard deviations and linear quartiles.

8. Conclusion and Future Work

This Design Science study asked how to govern human–AI collaborative delivery when agents act on external platforms that cannot undo or de-duplicate their actions. *Agentic Work Lab* and *PublishOps CrossCast* combine clear role boundaries between a human project manager and three AI workflows (GPT/Codex, Claude and DeepSeek) with four layers: an owner, stop-flag and due-time gate; plan-hash binding with SHA-256 media checks; a receipt state machine that saves an `UNKNOWN` state before a single final submission and never re-submits automatically; and independent read-back with human escalation. Over nine days, the field records show one recorded platform write and no recorded replay in a 69-record journal, an in-doubt Douyin submission held without re-submission, 30 SHA-256-recorded video versions, and a five-platform release whose evidence is kept per platform; 2,737 offline test functions exercise the corresponding code at commit `75b7abc`. These results support the mechanisms' design intent within the limits of a single case.

Future work follows from those limits: (1) record gate decisions and refused attempts, not only outcomes, so that suppression becomes measurable; (2) time bounded concurrent publication against sequential runs; (3) run the planned blinded identical-task comparison of models with logged costs; and (4) replicate the framework with other teams and accounts.

Data and Code Availability Statement

Source code, workflow verification scripts, the 69-record operational journal, the 55-record channel snapshot and the 30-entry media manifest are openly available at `https://github.com/whitecarp/agentic-work-lab` (data files as of commit `e3205d6`) and `https://github.com/whitecarp/publishops-crosscast` (evaluated code at commit `75b7abc`). The script `paper/reproduce_paper_tables.mjs` recomputes every figure reported here. `bili-cli` v1.2.1 (MIT) is bundled unchanged in *CrossCast*; API routes for YouTube, X and Instagram are distributed separately and lie outside the evaluated snapshot.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Adams, B., & McIntosh, S. (2016). Modern release engineering in a nutshell—Why researchers should care. In *Proceedings of the IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)* (pp. 78–90). IEEE.
2. Amershi, S., Weld, D., Vorvoreanu, M., et al. (2019). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (pp. 1–13). ACM.
3. Bainbridge, L. (1983). Ironies of automation. *Automatica*, 19(6), 775–779.
4. Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional.
5. Berente, N., Gu, B., Recker, J., & Santhanam, R. (2021). Managing artificial intelligence. *MIS Quarterly*, 45(3), 1433–1450.
6. Bianchi, M. J., Conforto, E. C., Rebentisch, E., Amaral, D. C., Rezende, S. O., & de Pádua, R. (2022). Recommendation of project management practices: A contribution to hybrid models. *IEEE Transactions on Engineering Management*, 69(6), 3558–3571.
7. Conforto, E. C., & Amaral, D. C. (2016). Agile project management and stage-gate model—A hybrid framework for technology-based companies. *Journal of Engineering and Technology Management*, 40, 1–14.
8. Cooper, R. G. (1990). Stage-gate systems: A new tool for managing new products. *Business Horizons*, 33(3), 44–54.
9. Flyvbjerg, B. (2006). Five misunderstandings about case-study research. *Qualitative Inquiry*, 12(2), 219–245.
10. Garcia-Molina, H., & Salem, K. (1987). Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data* (pp. 249–259). ACM.
11. Gray, J., & Reuter, A. (1992). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.
12. Gregor, S., & Hevner, A. R. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37(2), 337–355.
13. Helland, P. (2012). Idempotence is not a medical condition. *Communications of the ACM*, 55(5), 56–65.

14. Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.

15. Holzmann, V., Zitter, D., & Peshkess, S. (2022). The expectations of project managers from artificial intelligence: A Delphi study. *Project Management Journal*, 53(5), 438–455.

16. Hong, S., Zhuge, M., Chen, J., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*.

17. Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

18. Kerzner, H. (2022). *Project Management: A Systems Approach to Planning, Scheduling, and Controlling* (13th ed.). Wiley.

19. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

20. Leite, L., Rocha, C., Kon, F., Milojicic, D., & Meirelles, P. (2019). A survey of DevOps concepts and challenges. *ACM Computing Surveys*, 52(6), 1–35.

21. Müller, R., Locatelli, G., Holzmann, V., Nilsson, M., & Sagay, T. (2024). Artificial intelligence and project management: Empirical overview, state of the art, and guidelines for future research. *Project Management Journal*, 55(1), 9–15.

22. Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, 30(3), 286–297.

23. Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77.

24. Project Management Institute (PMI). (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* (7th ed.). PMI.

25. Qian, C., Liu, W., Liu, H., et al. (2024). ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)* (pp. 15174–15186).

26. Raisch, S., & Krakowski, S. (2021). Artificial intelligence and management: The automation–augmentation paradox. *Academy of Management Review*, 46(1), 192–210.

27. Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164.

28. Saltzer, J. H., Reed, D. P., & Clark, D. D. (1984). End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4), 277–288.

29. Schneider, F. B. (1990). Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4), 299–319.

30. Sein, M. K., Henfridsson, O., Purao, S., Rossi, M., & Lindgren, R. (2011). Action design research. *MIS Quarterly*, 35(1), 37–56.

31. Shahin, M., Ali Babar, M., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access*, 5, 3909–3943.

32. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 36.

33. Shneiderman, B. (2020). Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction*, 36(6), 495–504.

34. Too, E. G., & Weaver, P. (2014). The management of project management: A conceptual framework for project governance. *International Journal of Project Management*, 32(8), 1382–1394.

35. Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems*, 25(1), 77–89.

36. Wang, L., Ma, C., Feng, X., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345.

37. Whyte, J., Stasis, A., & Lindkvist, C. (2016). Managing change in the delivery of complex projects: Configuration management, asset information and 'big data'. *International Journal of Project Management*, 34(2), 339–351.

38. Wieringa, R. J. (2014). *Design Science Methodology for Information Systems and Software Engineering*. Springer.

39. Wieringa, R., & Daneva, M. (2015). Six strategies for generalizing software engineering theories. *Science of Computer Programming*, 101, 136–152.

40. Wu, Q., Bansal, G., Zhang, J., et al. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Proceedings of the Conference on Language Modeling (COLM)*.

41. Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.